

AI 모델 성능 평가 및 예측 결과 보고서 작성

- ****예측 결과를 보고서 형태로 정리"하기 위해 작성된 것으로, Transformer 모델 (또는 다른 모델)로 예측한 결과(predicted_stock.csv)**를 불러와, **평가 지표 (MAE, MSE, RMSE, MAPE 등)**를 계산하고, **매수/매도 판단**과 같은 간단한 분석 결과를 함께 제공하는 **최종 보고서**를 만듭니다.**

코드가 하는 일은 크게 다음과 같습니다:

1. **CSV 파일(predict된 결과) 로드**
2. **일주일 후 실제값 vs. 예측값 비교 → 평가 지표 산출**
3. **마지막 데이터 기준 상승 가능성 분석 → 상승률(%) 계산**
4. **매수/매도 추천 (STRONG BUY, BUY, SELL 등) & 분석 문구 추가**
5. **최종 테이블 정리해서 CSV 저장 & 콘솔 출력**

코드 작성 목적

- **Transformer 모델로 일주일 후 주가를 예측한 결과(predicted_stock.csv)가 이미 생성됨.**
- 그러나 이 예측 결과가 얼마나 정확한지, 그리고 앞으로 실제 매매 전략에 활용하자면 어떻게 해석해야 할지 분석 보고서가 필요.
- **본 코드는 그 예측 결과(실제 vs 예측)를 기준으로:**
 1. **평가 지표(MAE, MSE, RMSE, MAPE, Accuracy) 산출**
 2. **가장 최근 데이터 기준으로 "다음주에 오를지/내릴지"(Rise Probability) 분석**
 3. **간단한 매수/매도 추천(STRONG BUY, BUY, SELL)과 분석 문구(Analysis) 생성**
 4. 이를 합쳐 하나의 테이블로 출력 & CSV 저장 → **최종 보고서** 형태를 만든다.

간단히 말해, ****학습했던 Transformer 모델이 '잘 예측하고 있는지****를 지표로 확인하고, **"지금 사야 되나, 팔아야 되나?"** 같은 기초적인 투자 의사결정 힌트를 자동으로 주는 코드입니다.

라인별 상세 설명

(1) Google Drive Mount & 라이브러리

```
from google.colab import drive
```

```
drive.mount('/content/drive')
```

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

- Google Colab 환경에서 Drive를 붙임(**drive.mount('/content/drive')**)
- **pandas, numpy**: 데이터 분석, 배열 연산
- **mean_absolute_error, mean_squared_error**: 평가 지표 계산 함수.

(2) evaluate_predictions 함수

```
def evaluate_predictions(data, target_columns, forecast_horizon):
```

```
    """
```

```
    ...
```

```
    """
```

```
    metrics = []
```

```
    for col in target_columns:
```

```
        predicted_col = f'{col}_Predicted'
```

```
actual_col = f'{col}_Actual'
```

...

- **목적:** **predicted_stock.csv** 안에는 **QQQ ETF_Predicted**, ****QQQ ETF_Actual****처럼 예측값과 실제값이 있는데, 이를 이용해 **7일 뒤 예측의 정확성**을 측정.
- ****forecast_horizon=7****을 이용해서 실제값을 7일 뒤로 shift(미루기) → “오늘 예측 vs 7일 뒤 실제값”을 매칭.

```
predicted = data[predicted_col]
```

```
actual = data[actual_col].shift(-forecast_horizon)
```

```
valid_idx = ~predicted.isna() & ~actual.isna()
```

```
predicted = predicted[valid_idx]
```

```
actual = actual[valid_idx]
```

- 결측치(na)가 없는 구간만 필터링.

```
mae = mean_absolute_error(actual, predicted)
```

```
mse = mean_squared_error(actual, predicted)
```

```
rmse = mse ** 0.5
```

```
mape = (abs((actual - predicted) / actual).mean()) * 100
```

```
accuracy = 100 - mape
```

- **MAE:** 평균절댓값오차 → 단위가 실제 주가와 동일, “평균적으로 얼마만큼 틀렸는지”.
- **MSE:** 제곱오차평균 → 큰 오류가 있을 때 더 크게 반영.
- **RMSE:** MSE를 루트 씌운 값. 직관적으로 해석 가능.

- **MAPE**: 실제값 대비 예측값이 평균 몇 % 오차가 나는지.
- **Accuracy = 100 - MAPE**: MAPE가 10%면 Accuracy는 90%라고 단순 해석.

```
metrics.append({
    'Stock': col,
    'MAE': mae,
    'MSE': mse,
    'RMSE': rmse,
    'MAPE (%)': mape,
    'Accuracy (%)': accuracy
})
```

- 결과를 딕셔너리 형태로 기록해둠.
- 마지막에 ****pd.DataFrame(metrics)****로 표 형태 반환.

(3) analyze_rise_predictions 함수

```
def analyze_rise_predictions(data, target_columns):
    last_row = data.iloc[-1]
    results = []

    for col in target_columns:
        last_actual_price = last_row.get(f'{col}_Actual', np.nan)
        predicted_future_price = last_row.get(f'{col}_Predicted', np.nan)
        ...
```

- **목적:** 데이터프레임의 **가장 최근 행(마지막 날짜)** 를 보고,
 - 실제 증가(_**Actual**)와 모델이 예측한 7일 후 증가(_**Predicted**)를 비교.
 - **상승 여부(predicted_future_price > last_actual_price)**와 **상승률** 계산.

```
predicted_rise = predicted_future_price > last_actual_price
```

```
rise_probability = ((predicted_future_price - last_actual_price) / last_actual_price) * 100
```

- 예: 실재가 100, 예측이 105라면, **rise_probability = (105-100)/100 * 100 = 5%**.

Note: 여기서는 "마지막 행"만 사용하므로, "가장 최근 날짜 기준, 다음주에 오를 것으로 보이냐?"를 간단히 체크.

(4) Recommendation & Analysis

```
def generate_recommendation(row):

    rise_prob = row.get('Rise Probability (%)', 0)

    predicted_rise = row.get('Predicted Rise', False)

    if pd.isna(rise_prob) or pd.isna(predicted_rise):

        return "No Data"

    if predicted_rise and rise_prob > 0:

        if rise_prob > 2:

            return "STRONG BUY"

        else:
```

```
        return "BUY"

    else:

        return "SELL"
```

- **generate_recommendation:**

- 단순 로직 예시: 상승률이 2% 넘으면 "STRONG BUY", 0% 넘으면 "BUY", 아니면 "SELL".
- 실제 투자 로직에는 훨씬 많은 고려사항이 필요하겠지만, 여기서는 데모.

```
def generate_analysis(row):

    stock_name = row['Stock']

    rise_prob = row.get('Rise Probability (%)', 0)

    predicted_rise = row.get('Predicted Rise', False)

    if pd.isna(rise_prob) or pd.isna(predicted_rise):

        return f"{stock_name}: Not enough data"

    if predicted_rise:

        return f"{stock_name} is expected to rise by about {rise_prob:.2f}%. Consider buying or holding."

    else:

        return f"{stock_name} is expected to fall by about {-rise_prob:.2f}%. A cautious approach is recommended."
```

- **generate_analysis:**

- 한 줄짜리 문구. 예) "QQQ ETF is expected to rise by about 3.50%."

Consider buying or holding.”

- 간단한 해석 문구를 자동으로 붙여서 **보고서**처럼 만듦.

(5) 메인 로직

```
predicted_file_path = '/content/drive/My Drive/predicted_stock.csv'
```

```
data = pd.read_csv(predicted_file_path, parse_dates=['날짜'])
```

- **1) CSV 파일 로드**

- ****predicted_stock.csv****는 이미 Transformer 모델로 예측한 결과(실제 vs 예측)가 들어있는 파일.

```
target_columns = [ 'QQQ ETF' ]
```

```
forecast_horizon = 7
```

- 어떤 종목(칼럼)을 대상으로 할지(예: QQQ ETF), 7일 후 예측으로 설정.

3) Evaluate predictions

```
evaluation_results = evaluate_predictions(data, target_columns, forecast_horizon)
```

```
print("===== Evaluation Results =====")
```

```
print(evaluation_results)
```

- 위에서 정의한 **evaluate_predictions** 함수를 호출해, **MAE, MSE, RMSE, MAPE, Accuracy**를 계산.
- 콘솔에 출력.

4) Analyze future rise

```
rise_results = analyze_rise_predictions(data, target_columns)
print("===== Rise Predictions =====")
print(rise_results)
```

- 마지막 날짜 기준으로 **상승률**과 ****Predicted Rise(True/False)****를 표시.

5) Merge DataFrames

```
final_results = pd.merge(evaluation_results, rise_results, on='Stock', how='outer')
```

- 평가 지표(evaluation_results) + 마지막날 분석(rise_results) 통합.
- on='Stock' → "QQQ ETF" 등 종목 이름 기준으로 합침.

6) Sort by rise probability

```
final_results = final_results.sort_values(by='Rise Probability (%)', ascending=False)
```

- 예상 상승 확률이 가장 큰 종목부터 나열하려면 유용(현재는 한 종목만 있어도 향후 확장 가능).

7) Generate buy/sell recommendations and analysis

```
final_results['Recommendation'] = final_results.apply(generate_recommendation, axis=1)
```

```
final_results['Analysis'] = final_results.apply(generate_analysis, axis=1)
```

- 각각 함수 호출해 **매수/매도, 해석 문구**를 한 열씩 추가.

Reorder columns

```
column_order = [  
    'Stock',  
    'MAE', 'MSE', 'RMSE', 'MAPE (%)', 'Accuracy (%)',  
    'Last Actual Price', 'Predicted Future Price', 'Predicted Rise', 'Rise Probability (%)',  
    'Recommendation', 'Analysis'  
]
```

```
final_results = final_results[column_order]
```

- 보기 좋게 컬럼 순서를 재배치.

8) Save final results to CSV

```
final_output_path = '/content/drive/My Drive/final_stock_analysis.csv'
```

```
final_results.to_csv(final_output_path, index=False)
```

```
print(f"Final combined results saved to {final_output_path}")
```

- 최종 보고서 형태를 **final_stock_analysis.csv**로 저장.
- 이제 이 CSV를 열어보면, **종목 이름, 예측 정확도 지표, 마지막 실제/예측 가격,**

매수/매도 판단, 간단한 문구 등이 한 번에 나타남.

9) Print final report

```
print("===== Final Report =====")  
  
print(final_results.to_string(index=False))
```

- 콘솔에도 표시.
 - `**to_string(index=False)**`로 깔끔하게 표 형태 출력.
-

결론

- 이 코드는 `***예측이 끝난 후 최종 보고서***`를 자동 생성하는 목적입니다.
- **주요 기능:**
 1. **평가 지표**(MAE, MSE, RMSE, MAPE, Accuracy)로 **모델 성능**이 괜찮은지 체크.
 2. **가장 최근**(마지막 행) 기준으로 "다음주 오를 가능성"을 추정.
 3. **간단 추천 로직**으로 "STRONG BUY", "BUY", "SELL" 등 판단.
 4. **분석 문구**를 달아 최종 결과("QQQ ETF is expected to rise by...")로 **보고서** 작성.
- 이 로직을 통해, 학습한 모델(Transformer 등)이 제대로 예측했는지 한눈에 파악하고, **기본적인 투자 의사결정(매수/매도) 참고자료**를 얻을 수 있게 됩니다.

실제 투자에서는 당연히 더 복잡한 기준이 필요하지만, 본 예시는 **모델 예측을 정리해주는 자동 보고서** 생성의 시범 예시라고 볼 수 있습니다.